

## Laporan Hands-on 6

**Resource competition, deadlock, deadlock prevention**, serta **blocking synchronization** menjadi fokus utama pada sistem concurrent. Pembahasannya memperluas persoalan dari sekadar race condition menjadi situasi ketika beberapa thread tidak hanya berbagi data, tetapi juga **bersaing memperebutkan resource**. Di titik ini, masalah concurrency bisa berkembang menjadi sistem yang berhenti membuat progres atau membuat sebagian alur terus tertunda.

Seluruh eksperimen memperlihatkan bahwa pengelolaan resource dalam sistem concurrent tidak cukup hanya dengan membuat thread berjalan bersama. Sistem juga harus memastikan urutan akuisisi resource aman, thread tidak saling menunggu secara melingkar, dan mekanisme sinkronisasi benar-benar menggunakan blocking yang efisien, bukan busy waiting.

### Section 1 - Thread Concurrency dan Resource Competition

Ide dasar bahwa beberapa thread dapat berjalan secara independent sambil tetap berbagi resource sistem yang sama diperkenalkan terlebih dahulu. Tujuan utamanya adalah menunjukkan bahwa concurrency secara alami menghasilkan **interleaving execution**, yaitu keluaran dari beberapa thread muncul bergantian sesuai keputusan scheduler. Selama belum ada sinkronisasi eksplisit, urutan eksekusi tidak dapat dipastikan walaupun source code terlihat sederhana.

Implementasi lab menggunakan dua thread yang sama-sama menjalankan fungsi task, kemudian mencetak status iterasi sebanyak lima kali dengan jeda `sleep(1)`. Kedua thread dibuat hampir bersamaan, sehingga keluaran "Thread 1 is running iteration ..." dan "Thread 2 is running iteration ..." akan bercampur. Karena masing-masing thread tidak saling memblok atau memodifikasi shared data yang sensitif, tidak muncul kesalahan hasil. Namun sifat **interleaved output** tetap menunjukkan bahwa CPU time dibagi secara dinamis di antara thread-thread yang aktif.

Dari sudut pandang sistem operasi, pembahasan ini menegaskan bahwa concurrency pada level thread selalu berkaitan dengan kompetisi terhadap resource yang dapat dipakai ulang, seperti CPU time, memori, dan perangkat I/O. Meskipun belum ada deadlock atau starvation pada tahap ini, praktikum ini menunjukkan fondasi masalah tersebut: ketika banyak alur dieksekusi bersamaan, sistem harus menentukan siapa yang mendapat resource terlebih dahulu. Dengan demikian, unpredictability yang terlihat di output menjadi titik awal memahami persoalan sinkronisasi yang lebih serius.

### Ringkasan Eksperimen

| Komponen           | Informasi |
|--------------------|-----------|
| File praktikum     | lab1.c    |
| Jumlah thread      | 2 thread  |
| Iterasi per thread | 5 iterasi |

| Komponen             | Informasi                                          |
|----------------------|----------------------------------------------------|
| Jeda per iterasi     | sleep(1)                                           |
| Shared data sensitif | Tidak ada                                          |
| Fenomena utama       | Output dua thread saling terinterleaving           |
| Konsep utama         | Resource competition dan nondeterministic ordering |

### Kesimpulan Section 1

Thread yang berjalan secara concurrent akan berbagi CPU time dan menghasilkan urutan eksekusi yang tidak pasti. Walaupun belum menimbulkan kesalahan hasil, interleaving ini adalah gejala awal dari resource competition dalam sistem concurrent. Gejala awal tersebut menjadi fondasi penting untuk memahami mengapa pengelolaan resource harus dirancang dengan hati-hati.

## Section 2 - Simulasi Deadlock dengan Threads

Bentuk klasik **deadlock** muncul ketika dua thread sama-sama berhenti membuat progres karena masing-masing menunggu resource yang dipegang thread lain. Skenario yang dipakai menggunakan dua mutex, A dan B, serta dua thread yang mengunci keduanya dalam urutan berbeda. Thread pertama mengunci A lalu mencoba mengambil B, sementara thread kedua mengunci B lalu mencoba mengambil A.

Begitu kedua thread berhasil mengambil mutex pertamanya masing-masing, keduanya akan masuk ke keadaan saling menunggu. Thread 1 tidak dapat melanjutkan karena menunggu B, sedangkan Thread 2 tidak dapat melanjutkan karena menunggu A. Tidak ada thread yang mau atau bisa melepaskan resource pertamanya karena keduanya masih berharap memperoleh resource kedua. Inilah bentuk langsung dari **circular wait**, salah satu syarat penting terjadinya deadlock.

Dari perspektif sistem operasi, eksperimen ini sangat penting karena menunjukkan bahwa deadlock bukan sekadar program “lambat”, melainkan program yang **berhenti membuat progres tanpa batas waktu**. Walaupun semua thread masih ada, sistem praktis tidak lagi bergerak. Dengan melihat output terakhir yang muncul sebelum program freeze, mahasiswa dapat memahami bahwa resource allocation order sangat menentukan apakah concurrency tetap produktif atau justru berubah menjadi kebuntuan permanen.

### Ringkasan Eksperimen

| Komponen        | Informasi                                   |
|-----------------|---------------------------------------------|
| File praktikum  | Praktikum deadlock dengan dua mutex A dan B |
| Shared resource | Mutex A dan B                               |
| Thread 1        | Mengunci A lalu mencoba B                   |
| Thread 2        | Mengunci B lalu mencoba A                   |
| Fenomena utama  | Program freeze tanpa selesai                |

| Komponen                          | Informasi                                                  |
|-----------------------------------|------------------------------------------------------------|
| Syarat deadlock yang tampak jelas | Circular wait                                              |
| Konsep utama                      | Deadlock akibat urutan akuisisi resource yang bertentangan |

## Kesimpulan Section 2

Concurrency dapat berhenti total ketika thread mengambil resource dalam urutan yang saling bertentangan. Deadlock membuat semua pihak yang terlibat tetap hidup tetapi tidak bisa melanjutkan pekerjaan. Hal ini menegaskan bahwa urutan akuisisi resource merupakan aspek yang sangat krusial dalam desain sistem concurrent.

## Section 3 - Deadlock Prevention melalui Resource Ordering

Solusi konseptual terhadap deadlock terlihat melalui upaya **menghilangkan circular wait** dengan menetapkan urutan akuisisi resource yang konsisten. Kedua thread tetap membutuhkan mutex A dan B, tetapi sekarang keduanya diwajibkan selalu mengambil A terlebih dahulu, baru kemudian B. Dengan aturan yang sama untuk semua thread, dependency melingkar tidak dapat terbentuk.

Hasilnya sangat berbeda dari eksperimen sebelumnya. Walaupun kedua thread masih saling berbagi resource yang sama, program kini berjalan sampai selesai tanpa freeze. Salah satu thread mungkin tetap harus menunggu thread lain melepaskan A, tetapi penantian itu bersifat sementara dan tidak berkembang menjadi deadlock. Ini menunjukkan bahwa **waiting** sendiri belum tentu berbahaya; yang berbahaya adalah ketika waiting membentuk lingkaran ketergantungan yang tidak bisa diputus.

Dari sudut pandang sistem operasi, pembahasan ini menunjukkan bahwa deadlock sering kali lebih mudah dicegah lewat **aturan desain** daripada diatasi setelah terjadi. Resource ordering merupakan strategi yang banyak dipakai karena sederhana dan efektif: semua thread dipaksa mengikuti aturan global yang sama. Dengan begitu, sistem tetap concurrent, tetapi risiko kebuntuan dapat ditekan sejak awal.

## Ringkasan Eksperimen

| Komponen                 | Informasi                                                  |
|--------------------------|------------------------------------------------------------|
| File praktikum           | Implementasi <code>safe_thread</code> dengan mutex A dan B |
| Strategi utama           | Semua thread mengunci A lalu B                             |
| Shared resource          | Mutex A dan B                                              |
| Hasil utama              | Program selesai tanpa deadlock                             |
| Masalah yang dihilangkan | Circular wait                                              |
| Konsep utama             | Deadlock prevention via resource ordering                  |

## Kesimpulan Section 3

Deadlock dapat dicegah dengan aturan urutan resource yang konsisten. Meskipun thread tetap harus menunggu resource, penungguan itu tidak lagi berubah menjadi kebuntuan

permanen. Pola seperti ini menegaskan bahwa desain sinkronisasi yang baik sering kali dimulai dari kebijakan akuisisi resource yang sederhana tetapi konsisten.

## Section 4 - Dining Philosophers dengan Semaphores

Masalah klasik **Dining Philosophers** dibahas untuk menjelaskan hubungan antara resource sharing, deadlock, dan starvation. Lima philosopher diimplementasikan sebagai thread, sedangkan lima fork diwakili oleh semaphore. Setiap philosopher harus mengambil dua fork sebelum bisa makan. Pola ini menyerupai banyak situasi nyata di sistem operasi ketika beberapa eksekusi bersaing memperebutkan lebih dari satu resource sekaligus.

Implementasi sederhana pada lab ini membuat setiap philosopher mengambil fork kiri lebih dulu, lalu fork kanan. Secara logika, program terlihat wajar karena setiap thread melakukan langkah yang sama. Namun justru karena semuanya melakukan langkah yang identik, ada kemungkinan seluruh philosopher memegang satu fork dan sama-sama menunggu fork kedua. Dalam kondisi seperti itu, sistem masuk ke situasi yang sangat mirip dengan deadlock. Selain itu, tergantung keputusan scheduler, ada juga kemungkinan beberapa philosopher terus-menerus kalah cepat dan mengalami **starvation**.

Penting untuk melihat bahwa problem concurrency tidak selalu dapat dipahami hanya dari satu atau dua thread. Ketika jumlah thread bertambah dan pola kompetisi resource makin kompleks, desain sinkronisasi menjadi jauh lebih sulit. Dining Philosophers menegaskan bahwa semaphore dapat digunakan untuk memodelkan resource sharing, tetapi solusi naive belum tentu aman. Dengan demikian, contoh ini menghubungkan teori deadlock dengan salah satu contoh paling terkenal dalam sistem operasi.

### Ringkasan Eksperimen

| Komponen            | Informasi                                                   |
|---------------------|-------------------------------------------------------------|
| File praktikum      | Implementasi philosopher dengan <code>sem_t forks[N]</code> |
| Jumlah thread       | 5 philosopher                                               |
| Shared resource     | 5 fork                                                      |
| Mekanisme utama     | <code>sem_wait</code> dan <code>sem_post</code>             |
| Risiko yang diamati | Deadlock atau starvation                                    |
| Pola resource       | Setiap thread butuh dua resource sekaligus                  |
| Konsep utama        | Dining Philosophers, deadlock, starvation                   |

### Kesimpulan Section 4

Concurrency menjadi jauh lebih rumit ketika banyak thread memperebutkan beberapa resource sekaligus. Pada implementasi yang sederhana, sistem dapat jatuh ke deadlock atau menyebabkan starvation pada sebagian thread. Kondisi ini memperjelas bahwa sinkronisasi yang terlihat benar secara lokal belum tentu aman secara global.

## Section 5 - Blocking Synchronization menggunakan Semaphores

Konsep **blocking synchronization** ditekankan pada bagian penutup, yaitu ketika thread menunggu secara efisien alih-alih terus melakukan busy waiting. Pola producer-consumer kembali dipakai, tetapi sekarang fokus utamanya bukan sekadar pada benar atau salahnya hasil, melainkan pada cara thread diblok dan dibangun kembali berdasarkan ketersediaan slot buffer atau item. Producer menunggu pada semaphore empty ketika buffer penuh, sedangkan consumer menunggu pada semaphore full ketika buffer kosong.

Kombinasi tiga semaphore, yaitu empty, full, dan mutex, menunjukkan pembagian peran sinkronisasi yang jelas. empty mengatur ruang kosong buffer, full mengatur jumlah item tersedia, dan mutex bertindak sebagai binary semaphore untuk melindungi critical section saat array buffer diakses. Ketika salah satu syarat belum terpenuhi, thread tidak terus-menerus mengecek kondisi, melainkan benar-benar diblok oleh kernel sampai semaphore memberi sinyal bahwa resource telah tersedia.

Dari sudut pandang sistem operasi, inilah bentuk sinkronisasi yang jauh lebih efisien dibanding spinning. CPU tidak terbuang untuk loop pengecekan yang tidak produktif, dan thread baru dijadwalkan kembali ketika memang dapat melanjutkan pekerjaan. Dengan demikian, eksperimen ini menekankan bahwa sinkronisasi yang baik harus menjaga **correctness, ordering**, sekaligus **efisiensi penggunaan CPU**.

### Ringkasan Eksperimen

| Komponen        | Informasi                                                     |
|-----------------|---------------------------------------------------------------|
| File praktikum  | Producer-consumer dengan semaphore                            |
| Shared resource | Buffer ukuran SIZE = 5                                        |
| Semaphore       | empty, full, mutex                                            |
| Thread producer | Menambah item ke buffer                                       |
| Thread consumer | Mengurangi item dari buffer                                   |
| Fenomena utama  | Thread diblok dan dibangun sesuai kondisi buffer              |
| Konsep utama    | Blocking synchronization dan efisiensi dibanding busy waiting |

### Kesimpulan Section 5

Semaphore dapat dipakai untuk membangun sinkronisasi yang tidak hanya aman, tetapi juga efisien. Thread tidak perlu terus aktif memeriksa kondisi buffer, karena kernel dapat memblokirnya sampai resource benar-benar tersedia. Hal ini menegaskan bahwa desain concurrency yang baik harus memperhatikan pemakaian CPU selain correctness hasil.

### Penutup

Berdasarkan seluruh eksperimen pada praktikum ini, dapat disimpulkan bahwa pengelolaan resource pada program concurrent merupakan persoalan yang jauh lebih luas daripada sekadar membiarkan banyak thread berjalan bersama. Resource competition

dapat menimbulkan nondeterministic behavior, deadlock dapat menghentikan progres sistem sepenuhnya, starvation dapat membuat sebagian thread terus-menerus tertunda, dan desain sinkronisasi yang buruk dapat memboroskan CPU. Oleh karena itu, concurrency membutuhkan strategi pengaturan resource yang matang.

Secara keseluruhan, praktikum ini memperlihatkan tiga pelajaran besar. Pertama, urutan pengambilan resource sangat menentukan apakah program tetap berjalan atau justru macet. Kedua, problem klasik seperti Dining Philosophers menunjukkan bahwa concurrency global sering lebih rumit daripada yang terlihat dari perilaku lokal tiap thread. Ketiga, mekanisme blocking synchronization seperti semaphore yang dirancang dengan baik dapat menjaga correctness sekaligus efisiensi. Dengan demikian, praktikum ini melengkapi pemahaman tentang concurrency dari sisi **kompetisi resource, kebuntuan, dan desain sinkronisasi yang efisien.**